

AmazonHelp AI Support Agent – Report

By – anand kumar

Report: AmazonHelp AI Support Agent

1. Problem framing

What “good” means for this agent, for this brand:

For AmazonHelp specifically, “good” is not “correctly classify every tweet.” It’s:

- Never auto-handle something that could cause real harm if wrong (account security, unauthorized charges) — recall on escalation for these categories matters more than overall accuracy.
- A drafted reply that’s *safe to show a human reviewer as a starting point* even when it’s imperfect — it should never invent order numbers, refund amounts, or policy commitments not grounded in the actual message or precedent.
- Correctly identifying non-support noise (spam/irrelevant mentions) so a human isn’t wasting time triaging tweets that were never real support requests in the first place.

What I chose not to build:

- Multi-turn conversation state / follow-up tracking — the agent handles a single incoming message in isolation. A real deployment would need thread memory; out of scope for a 2-3 day take-home.
 - Automatic sending of replies — the agent always produces a *draft*, even when “auto-handle” is decided; a human-in-the-loop send step is assumed to exist downstream.
 - Fine-tuning a classifier — used few-shot prompting instead of fine-tuning a smaller supervised model on Banking77 or hand-labeled data, because the golden set (267 examples) is too small to fine-tune on responsibly, and few-shot prompting is more directly explainable in a live review.
 - Non-English support — the classifier, escalation keyword list, and reply drafter are all English-only; non-English messages (~17% of the raw AmazonHelp data) are filtered out before both retrieval grounding and golden-set sampling rather than handled with unpredictable results.
-

2. Results vs. baselines

System	Intent Accuracy	Intent Macro-F1	Escalation F1	Missed-Escalation Rate
Trivial baseline	0.326	0.055	0.000	0.210
Simple baseline	0.333	0.244	0.000	0.210
LLM agent	0.772	0.663	0.197	0.184

(Full precision/recall breakdown is in `eval/results.json`.)

Intent classification: the LLM agent clearly outperforms both baselines — 77.2% accuracy and 0.663 macro-F1 vs. ~33% accuracy and 0.055-0.244 macro-F1 for the baselines. The trivial baseline’s 32.6% accuracy exactly matches the majority class’s share of the golden set (`late_or_missing_delivery` at 87/267), which is a useful sanity check that the metric is computing correctly.

Escalation: the LLM agent is the only system that catches ANY true escalations at all — both baselines score exactly 0.000 F1 (the trivial baseline never escalates by design; the simple baseline’s “escalate only if no similar precedent found” heuristic almost never fires given how large the retrieval corpus is, 111,570 pairs). But 0.197 F1 is still weak in absolute terms: working out the underlying confusion matrix, **the agent only catches about 12.5% of examples that should have been escalated** (7 of 56 true-escalate cases), though when it does escalate, it’s reasonably precise (~46%). This is the single most important finding in this report — see Failure Analysis below.

Reply quality (LLM-as-judge, mean overall score 1-5): 3.75

This sits in a reasonable-but-not-excellent range — not embarrassing, but not a score that should be taken at face value without the judge-agreement check below, since 3.75 alone doesn’t say whether the judge is calibrated similarly to a human reader.

Judge-human agreement (on N=40 examples):

- Exact agreement rate: 45.0%
- Agreement within 1 point: 82.5%
- Mean absolute difference: 0.72
- Interpretation: 82.5% agreement within 1 point is reasonable enough to trust the judge for *relative* comparisons (e.g., “system A’s replies score higher than system B’s on average”) but not for *absolute* pass/fail gating of individual replies without spot-checks — a 45% exact match means the judge and a human reader agree on the precise score less than half the time. Looking at the biggest disagreements specifically: 4 of the 5 largest gaps had the judge scoring *higher* than the human rater (e.g. judge=5 vs. human=3), suggesting a mild leniency bias in the judge toward borderline-acceptable replies rather than a random-noise disagreement pattern.

3. Failure analysis — top 5 failure modes

Pulled directly from `eval/results_checkpoint.jsonl` cross-referenced against `eval/golden_set_labeled.csv` (script: `eval/extract_failures.py`).

Failure mode 1: No mechanism for “repeated failure” escalation signals

- Examples:
 - `ex_0012`: *“My last 5 parcels have been left in an unknown location after I’ve repeatedly reported both on the phone and online chat.”* → classified `general_complaint`, `escalate=False`, `reason=auto_handle:confident_intent_with_precedent`
 - `ex_0020`: *“I am sick and tired of twitting, mailing and calling you folks. You respond... but fail to resolve the problem.”* → same auto-handle outcome
 - `ex_0031`: *“My case has been closed without solution... no action taken over courier guy who change my product.”* → same outcome
- What went wrong: all three describe a *pattern* of prior failed contact attempts, which is exactly the kind of case a human should see. But the escalation rule set (`src/escalation.py`) only checks: hard keywords, `account_access` intent, classifier confidence, and retrieval precedent similarity. None of these detect “this is not the first time I’ve asked.”
- Hypothesis: the classifier is confident (these are linguistically ordinary complaints) and retrieval easily finds similar-sounding precedent (common phrasing), so both of the two “soft” escalation triggers pass — there’s no third trigger for repeated-contact language at all. This affected every single missed escalation observed (100% of the 8 sampled).

Failure mode 2: Financial-harm language not in the keyword list

- Examples:
 - `ex_0007`: *“ndtv has cheated me for an amount of rs 35000... Customer care blocked”* → classified `billing_dispute`, not escalated
 - `ex_0009`: *“it’s coming up on my bank a/c that I’ve been charged twice”* → classified `billing_dispute`, not escalated
 - `ex_0040`: *“Why are you charging me 200 dollars for shipping for something that costs 30 dollars?”* → classified `billing_dispute`, not escalated
- What went wrong: the classifier correctly identified all three as billing disputes — the escalation layer is what failed. `billing_dispute` is not in `ALWAYS_ESCALATE_INTENTS` (only `account_access` is), and none of these messages contain the literal keywords in `HIGH_RISK_KEYWORDS` (`fraud`, `unauthorized`, `lawyer`, `BBB`, etc.) — real customers say “cheated,” “charged twice,” or just describe the discrepancy, not the legal-sounding trigger words the keyword list expects.
- Hypothesis: the keyword list was built by guessing likely phrases rather than sampling how people actually describe billing harm in this dataset. A better design

would either escalate `billing_dispute` by default above some amount/severity threshold, or expand the keyword set based on real examples like these.

Failure mode 3: Fraud-adjacent situations without fraud-sounding words

- Example: `ex_0010`: *"I bought from a seller I think is fake... HELP ME!!!"* → classified `wrong_or_damaged_item`, not escalated (true intent: `refund_or_return`)
- What went wrong: two failures stacked — wrong intent AND missed escalation. "Fake seller" is a marketplace-fraud signal that isn't a literal keyword match, and the message's actual need (help filing a claim / getting a refund) got classified as a damaged-item complaint instead.
- Hypothesis: the classifier's few-shot examples likely didn't cover "suspected counterfeit/fake seller" as its own pattern distinct from a physically damaged item — both can sound similar in isolation.

Failure mode 4: Intent confusion on compound or ambiguous messages

- Examples:
 - `ex_0013`: *"jacking up price and then discounting... Price is 6000. You show 6490."* → predicted `general_complaint`, true `billing_dispute`
 - `ex_0019`: *"I have an order that want's me to revise payment settings but doesn't give me an option to do so"* → predicted `account_access`, true `billing_dispute`
 - `ex_0004`: *"how can a regular user get a new laptop delivered by Sat and a prime user (me) only by Monday?!"* → predicted `general_complaint`, true `product_question`
- What went wrong: these are genuinely borderline cases — `ex_0019` in particular could reasonably be argued either way (payment settings touch both account and billing). Not all of these are model failures in the strict sense; some reflect real ambiguity in a 9-category taxonomy applied to messages that blend two concerns.
- Hypothesis: a small number of intent categories inherently overlap (billing vs. account access; general complaint vs. a specific intent expressed angrily) — some disagreement here is a taxonomy-design limitation, not purely a classifier weakness.

Failure mode 5: Garbled/low-signal text produces unstable classification

- Example: `ex_0022`: *"Amzl and today I've never got teen anything from you guys pass 2"* → predicted `late_or_missing_delivery`, true `general_complaint` (I had flagged this exact example during labeling as "low confidence, garbled text")
- What went wrong: the message itself is barely parseable English (likely a typo-laden or autocorrect-mangled tweet). The classifier still confidently produced an intent rather than signaling low confidence.
- Hypothesis: this is as much a data-quality issue as a model issue — some fraction of real Twitter support messages are simply too garbled to classify reliably, and the pipeline currently has no "message unparseable" fallback path.

Summary pattern across all 5 modes: four of five failure modes are *escalation* failures or *intent* failures caused by the escalation/intent design not anticipating a real-world phrasing pattern — not random model noise. This is actionable: modes 1-3 point to concrete, specific improvements (a repeated-contact detector, a broader/learned keyword set, better fraud-vs-damage few-shot examples) rather than “the model needs to be better” in the abstract.

4. What is misleading about my headline number?

This section is mandatory — be honest here, not defensive.

Confirmed, concrete candidates for THIS project (not hypothetical — these are real facts about how this specific golden set and pipeline were built):

- **Golden set size (267) is small relative to 9 intent classes, and the split is heavily uneven** — `late_or_missing_delivery` (87) and `general_complaint` (62) dominate, while `order_status` has only 3 examples and `spam_or_irrelevant` has only 1. Macro-F1 will be substantially influenced by performance on the well-represented classes; per-class F1 for the rare ones is not statistically meaningful at this N.
- **The golden set was labeled by a single labeler (me), with an AI assistant proposing labels I reviewed rather than an independent second human cross-checking them** — no inter-labeler agreement was computed because there was no second labeler. Escalation calls in particular involved real judgment (e.g., “does 3 unresolved contact attempts count as escalate-worthy?”) that a different labeler, or Hiver’s own support team, might draw differently.
- **The golden set was sampled from data pre-filtered by my own cleaning heuristics** (language detection + fragment removal), not the raw distribution — roughly 28% of the original AmazonHelp pairs were removed before sampling even began. If the real deployment would see that excluded 28% (non-English customers, or messages arriving without full thread context), this golden set doesn’t measure performance on them at all.
- **The LLM judge (gpt-oss-120b) and the reply-drafting model (gpt-oss-20b) are from the same model family** — even with a size gap, shared training data and stylistic tendencies could mean the judge is systematically more lenient toward this family’s phrasing than an independent judge (e.g., a different provider’s model) would be.
- **Escalation ground truth is itself a judgment call I made**, not an objective label — reasonable people (or Hiver’s own support team) might draw the escalation line differently, especially for `general_complaint`.
- **Evaluated on a subsample of a subsample** — the Kaggle dataset is already a filtered/anonymized public release, and this project evaluates a further cleaned subsample of AmazonHelp threads specifically. Real-world message distribution and edge-case frequency may differ.

- **The escalation F1 of 0.197, while much better than either baseline's 0.000, hides a severe recall problem** — 0.197 sounds like “somewhat working,” but the underlying numbers show the agent catches only ~12.5% of examples that genuinely needed escalation. A reader skimming just the F1 column could easily miss that this system would, in production, let the large majority of trust-and-safety-relevant messages through unescalated. This is the single most important “misleading number” in this report if not read carefully.
 - **The 77.2% intent accuracy is measured on a golden set already filtered to be classifiable** — fragments, non-English messages, and (per the labeling notes) some garbled/low-signal text were excluded before sampling. Real-world accuracy on the full unfiltered stream would likely be lower, since the excluded ~28% skews toward exactly the messages a classifier would struggle with.
-

5. What I'd do with one more week

Directly motivated by the failure analysis above, in priority order:

1. **Add a repeated-contact/pattern-of-failure signal to escalation** — this alone was responsible for the largest share of missed escalations observed. Could be as simple as a keyword/phrase expansion (“again”, “still”, “repeatedly”, “X times”, “weeks”) combined with a lower confidence bar when such phrases are present, or as involved as detecting multi-tweet threads from the same author.
 2. **Expand or replace the hard-coded escalation keyword list with one derived from real examples** — the current list was authored by guessing likely phrases; failure mode 2 shows real customers describe billing harm (“cheated”, “charged twice”) without using any of the listed words. Consider also just making `billing_dispute` escalate by default above a stated threshold rather than relying on keywords at all.
 3. **Second independent labeler for at least a subset of the golden set**, to compute real inter-labeler agreement — right now escalation ground truth reflects only my own judgment calls.
 4. **Add multi-turn thread context** instead of collapsing to first-message classification — several failure examples (e.g. `ex_0012`'s “repeatedly reported”) reference prior contact that a real thread-aware system would actually have visibility into instead of inferring from language alone.
 5. **Fine-tune a small classifier on a larger labeled set** and compare against the few-shot LLM approach, once more labeled data exists.
 6. **Build a regression test suite from these exact failure examples** so future changes to the escalation logic can be checked against known past failures, and add active learning to prioritize which new messages most need human labels next.
-

Decision log

Non-obvious decisions and why, in the order they came up:

1. **Chose AmazonHelp over a smaller brand** — enough volume and intent diversity to fill a 200-example golden set without running out of real variety per intent.
2. **Collapsed multi-turn threads to (first customer message, first brand reply) pairs** for retrieval grounding — simpler and more tractable than full thread modeling for a 2-3 day project; documented as an explicit scope cut, not an oversight.
3. **Used an explicit rule table for escalation, not an LLM judgment call** — escalation is a trust/safety boundary; a rule table is auditable and testable in a way an LLM's escalate/don't-escalate call isn't.
4. **Kept the intent taxonomy to 9 classes**, derived from skimming real data rather than guessing upfront — a larger taxonomy would fragment an already-small golden set into unreliable per-class metrics.
5. **Used word-boundary regex for high-risk keyword matching, not substring matching** — caught during testing that substring matching on “sue” false-positives on ordinary words like “issue.” (See `tests/test_agent_wiring.py::test_no_precedent_forces_escalation`, which is what surfaced this.)
6. **Chose Groq-hosted open-weight models (openai/gpt-oss-20b for classification/drafting, openai/gpt-oss-120b as judge) over a proprietary API** — fast/cheap enough for iteration within the time budget, and staying open-weight keeps the “you may use any LLM API or open model” framing honest even though inference is hosted, not local. Note: originally planned to use Llama 3.1/3.3, but Groq's actually available free-tier model lineup at the time of building this had shifted to the gpt-oss/Qwen family — checked via the live `client.models.list()` call against my own account rather than trusting documentation that could be stale, since model availability on hosted inference platforms changes over time.
7. **Used a stronger model (gpt-oss-120b) as the judge than the model doing classification/generation (gpt-oss-20b)** — a same-or-weaker judge model risks systematically favoring its own generation patterns.
8. **Built the retrieval index with sentence-transformers + brute-force cosine similarity, no vector DB** — at subsample scale (hundreds of thousands, not millions, of pairs), a vector DB adds complexity with no real latency or recall benefit, and is harder to explain/debug live.
9. **Every message gets a draft reply, even ones flagged for escalation** — a human reviewer benefits from a starting point rather than a blank page; changed the framing from “agent replies or doesn't” to “agent always drafts, routing decides who acts on it.”
10. **Reported macro-F1 as the headline classification metric, not raw accuracy** — raw accuracy would be dominated by whichever intent is most frequent in the golden set and would hide poor performance on rarer but important intents like `account_access`.
11. **Reported escalation recall and precision separately, not just F1** — missing an escalation that should have happened (false negative) is a materially different kind

of error than an unnecessary escalation (false positive), and collapsing them into one F1 number obscures which kind of error the system is actually making.

12. **Added an explicit data-cleaning step (language + fragment filtering) before both golden-set sampling and retrieval grounding** — an early, unfiltered sample showed ~17% non-English messages (the classifier, escalation keyword list, and reply drafter are all English-only) and ~11% messages that were mid-thread fragments (“UPS”, “did all of those already”) rather than standalone requests. Filtering both before sampling, rather than filtering the golden set alone, keeps the retrieval index and the eval set consistent with each other.
13. **Language detection uses a probability threshold via `detect_langs()`, not `langdetect`’s single top-1 `detect()` guess** — direct testing showed `detect()` misclassified a clearly-English sentence as Danish once a long numeric order ID was involved, because `langdetect`’s top-1 guess is unstable on short/ambiguous text. Also strip long digit sequences (order/tracking numbers) before detection for the same reason. This is a real, acknowledged limitation of automated language detection — a small number of heavily-abbreviated English messages (“stil nt dlvrđ vpoor srvc”) still get misclassified as non-English and dropped; not fully fixable without a much heavier model.
14. **High-risk escalation keywords are matched with word boundaries (`\bsue\b`), not substring matching** — substring matching on “sue” false-positived on ordinary words like “issue” and “pursue,” caught by an automated test rather than manual inspection (`tests/test_agent_wiring.py`).
15. **`openai/gpt-oss-20b/120b` are reasoning models that spend part of their token budget on hidden “thinking” tokens before the visible answer** — an initial `max_tokens=150` budget for reply drafting was entirely consumed by reasoning, returning an empty reply. Fixed by raising the budget, setting `reasoning_effort="low"`, and adding an automatic retry-with-larger-budget when a response comes back empty with `finish_reason="length"`, instead of silently returning “”.
16. **The evaluation run is checkpointed to disk per-example, resumable on restart** — Groq’s free tier has a hard daily token quota; hitting it mid-run initially would have meant losing all prior progress and re-burning quota redoing completed examples. Each result is written to a `.jsonl` file immediately and skipped on the next run if already present.
17. **Switched `CLASSIFY_MODEL` from `openai/gpt-oss-20b` to `qwen/qwen3.6-27b` partway through the project and re-ran the full evaluation from scratch, rather than mixing two models’ outputs in one result set** — hit Groq’s daily token quota on `gpt-oss-20b` with a multi-day wait to fully reset given the token volume needed; switching to a different model (separate quota bucket) let the eval finish same-day. Restarting cleanly rather than splicing together a partial `gpt-oss-20b` run with a partial Qwen run keeps every reported number attributable to one consistent model, at the cost of some earlier progress.
18. **Qwen3 models require `reasoning_format="hidden"` when JSON mode is requested, and only accept `reasoning_effort="none"/"default"` (never “low”/“medium”/“high” like `gpt-oss`)** — without this, Qwen’s default output

embeds literal `<think>...</think>` tags inside the JSON response, which Groq's own JSON-mode validator rejects outright with a 400 error (Failed to validate JSON), distinct from the silent-empty-content failure mode seen with gpt-oss. Found by hitting the actual error mid-run; fixed with model-family-specific parameter handling in `llm_client.py` rather than a one-off patch.

Golden set sampling & labeling note

- **Source data:** sampled from a cleaned subset of the AmazonHelp portion of the Kaggle Customer Support on Twitter dataset. Of 154,976 raw (customer message, brand reply) pairs reconstructed from the raw data, 138,329 remained after removing non-standalone fragments (messages under 4 words, or matching a short blacklist of acknowledgement phrases like “thanks”, “ok”, “UPS” — these are almost always the 2nd/3rd message in a thread and don't carry standalone intent), and 111,570 remained after additionally removing non-English messages (detected via `langdetect` with a probability threshold, not strict top-1 detection — see decision log). All golden set sampling was done from this cleaned 111,570-row pool, not the raw 154,976.
 - **Sampling method:** 300 messages drawn via `pandas.sample(random_state=1)` — a random, not explicitly stratified, sample from the cleaned pool. After labeling, an additional ~33 rows were dropped on manual review for still being fragment-like, ambiguous, or pure positive feedback with no actionable support intent (the automated fragment filter is a heuristic and doesn't catch every case) — final golden set size: 267.
 - **Labeling:** labeled by a single labeler (me), with an AI assistant (Claude) proposing intent/escalation labels for each message using a documented rule pattern (e.g., literal “fraud”/“lawyer”/“BBB” mentions and account-access issues always escalate; repeated-failure patterns — 3+ contact attempts, multi-week unresolved issues — escalate even for otherwise-routine intents; single instances of anger/profanity alone do NOT escalate). I reviewed and could override every suggested label. This is a real limitation, not hidden: no independent second labeler cross-checked these labels — see “misleading headline number” section.
 - **Intent distribution in the final 267:** heavily skewed toward `late_or_missing_delivery` (87) and `general_complaint` (62), with `order_status` (3) and `spam_or_irrelevant` (1) having too few examples for a reliable per-class metric — this reflects what's actually common in the real AmazonHelp Twitter traffic, not an artificial balancing choice, but it does mean per-class F1 for the rare intents should be read with low confidence.
 - **Escalation distribution:** 56 TRUE (~21%) / 211 FALSE (~79%) — a reasonable, non-degenerate split.
 - **Excluded from the source pool entirely:** non-English tweets, and messages that don't stand alone without earlier thread context (see cleaning methodology above) — both documented, stated scope cuts.
-

Citations / borrowed material

- Dataset: Kaggle `thoughtvector/customer-support-on-twitter` (<https://www.kaggle.com/datasets/thoughtvector/customer-support-on-twitter>)
- Retrieval embedding model: `sentence-transformers/all-MiniLM-L6-v2` (Reimers & Gurevych, `sentence-transformers` library)
- Language detection: `langdetect` Python library (port of Google's language-detection library)
- LLM inference: Groq API serving Qwen's open-weight `qwen3.6-27b` model (classification + reply drafting) and OpenAI's open-weight `gpt-oss-120b` (LLM-as-judge) — see decision log entries 6-7, 17-18 for why two different model families ended up in the final configuration